Process Management

Sistemas de Computadores

Departamento de Engenharia Informática
Instituto Superior de Engenharia do Porto

Luís Nogueira (lmn@isep.ipp.pt)

The process abstraction

- **Most modern OSes include the notion of a process**
  - Frequently described as "*an instance of a program in execution*"
  - The primary means by which multiprogramming/multitasking is offered in modern operating systems

- **A process is more than code**
  - The program's instructions (aka. the "program text")
  - CPU state for the process (program counter, registers, flags, …)
  - Memory state for the process
  - Other resources being used by the process

- **Multiple processes may execute the same program concurrently**

Traditional view of a process

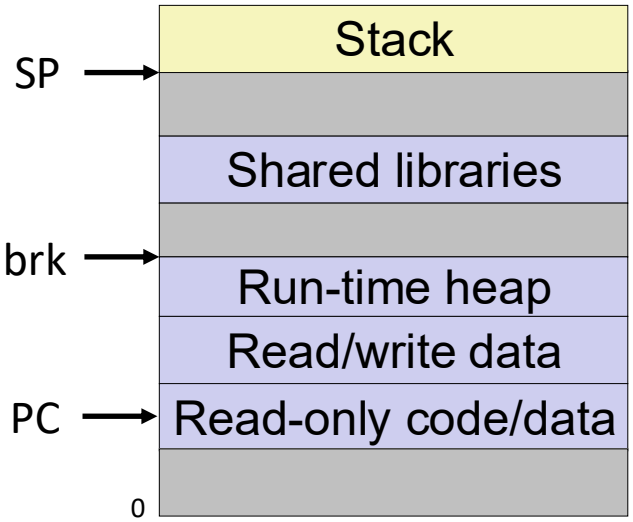
- **Process = process context + code, data, heap and stack**

Process context

Program context:
- Data registers
- Condition codes
- Stack pointer (SP)
- Program counter (PC)

Kernel context:
- VM structures
- Descriptor table
- brk pointer

Code, data, heap and stack



OS process management

- **Process identification and representation**
  - Maintain internal structures to track processes
- **Process creation and termination**
  - Manage user and system processes dynamically
- **CPU scheduling**
  - Allocate processor time efficiently among processes
- **Inter-process communication and synchronization**
  - Enable coordination and data exchange between processes

Process identification

- **A process is uniquely identified by a Process ID (PID), assigned by the OS**

- **PIDs are typically allocated sequentially until the maximum value of the data type is reached**
  - PID allocation is OS-dependent and that recycling policies are implementation-specific

The `fork` function

```
#include <sys/types.h>
#include <unistd.h>

pid_t fork();
```

- **A new process is created by duplicating the calling process**
  - Except for the `init` (`systemd`) process, whose PID is 1, all other processes came into existence because some other process created them

- **Processes can be created:**
  - By the OS (e.g., system startup, scheduled tasks)
  - By a user (e.g., launching an application)
  - By another process (e.g., spawning child processes)

Process hierarchy

- **The process that calls `fork()` is known as the parent process**
- **The newly created process is called the child process**
  - We call all processes created by the same process **siblings**
- **Both parent and child processes can create additional processes, forming a process tree**
  - Each process has exactly one parent but can have zero or more child processes

Process hierarchy

```
$ pstree | more

init-+-acpid
|-avahi-daemon---avahi-daemon
|-bonobo-activati---{bonobo-activati}
|-cron
|-cupsd
|-gdm---gdm-+-Xorg
|      `-x-session-manag-+-gnome-panel---{gnome-panel}
|                        |-gnome-settings--+-pulseaudio-+-gconf-helper
|                        |                 |           `-2*[{pulseaudio}]
|                        |-konsole---3*[bash]
|                        |-metacity
|                        |-ssh-agent
|-getty
|-konsole-+-2*[bash]
|         |-bash---vim
…
```

Process creation

- **When creating a process, both the parent and child processes share the same memory for the code**
  - This is because the code is read-only
- **Also, the data segment (global variables, heap, and stack) is initially shared between processes**
  - If a process attempts to modify the data, a separate copy of that data is created for the modifying process
  - This ensures that changes made by one process do not affect the other
  - This “copy only when necessary” behavior is known as Copy-on-Write (COW)

Process creation

- **After `fork()`, both processes execute the same code, starting from the instruction immediately after the `fork()` call**

- **The execution order is non-deterministic**
  - The OS scheduler determines which process runs at any given moment
  - Parent and child are **independent** processes that compete for CPU time

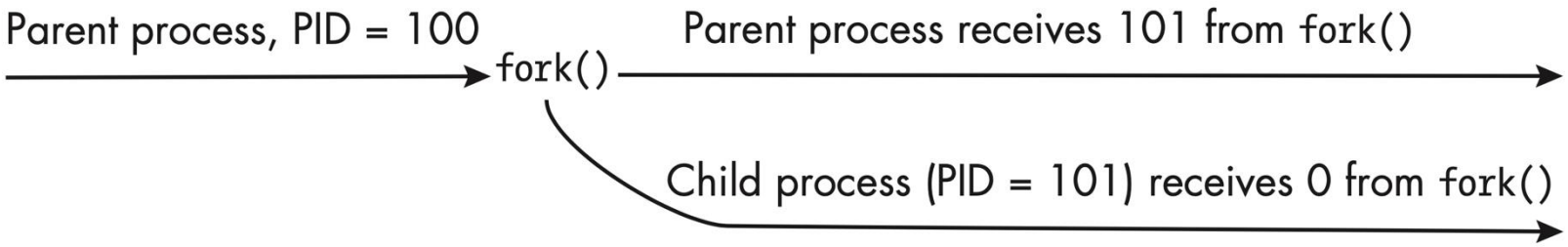
- **A mechanism is needed to distinguish between the parent and child after `fork()`**

`fork()` return values

- **-1, in case of error**
  - It was not possible to create the child, so a single process still exists
- **On success, the different values received by parent and child allows the separation of the code each one will execute**
- **The PID of the child (> 0) is returned to the parent process**
  - Parent may need this PID to communicate or synchronize with the child
- **Zero is returned to the child process**
  - Allowing it to distinguish itself from the parent

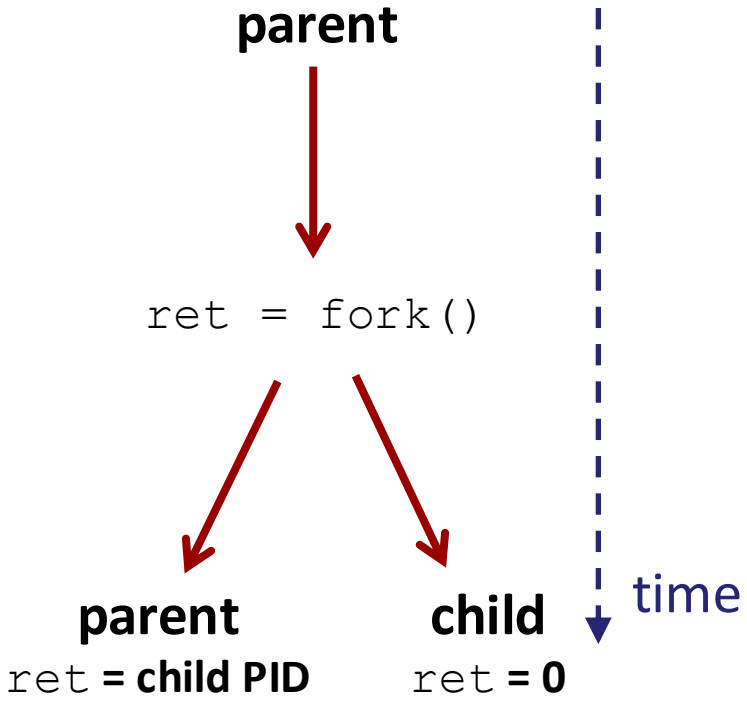
Example – `fork()` return values

```
pid = fork();

if(pid == -1){
   perror("Fork failed");
   exit(1);
}

if(pid > 0){
   printf("Luke, I'm your father\n");
}
else{
   printf("I am the child\n");
}

printf("Who executes this line?");
```



The `fork` function

- **`fork()` is a unique function!**
- **Called once, returns twice**
  - Return value in parent is the PID of the newly created child
  - Return value in the child is zero
- **Two loosely related processes executing alongside one another**



Changes after `fork`

- In the child process, variables start with the same values as in the parent at the moment `fork()` is executed

- However, any modifications made in either process after `fork()` are independent and do not affect the other process

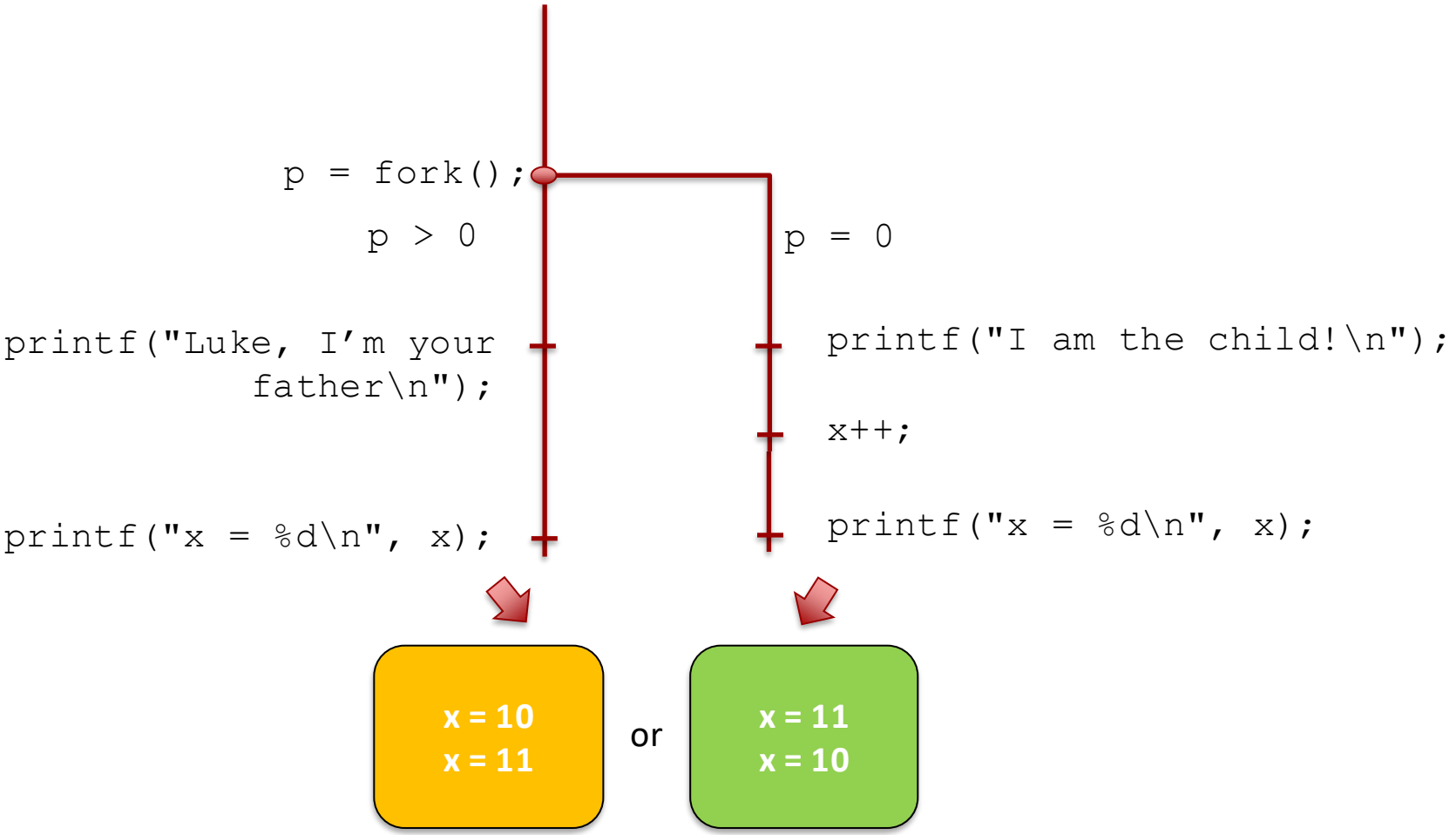
Example – changes after `fork`

```
int x = 10;

p = fork();

if(p > 0){
        printf("Luke, I'm your father\n");
}
else{
        printf("I am the child\n");
        /* child changes its copy of x */
        x++;
}

printf("x = %d\n", x);
```

Concurrent execution

- **Only one process may, at a given instant, be executing in a core of a CPU…**
- **… but multiple processes can be loaded to memory and be executed concurrently**
- **The goal of the OS is to have always a process executing, maximizing resource usage**
  - Decision based in metrics of performance and load (**scheduling**)
  - Scheduling will be discussed in detail in lecture classes later in the semester

Concurrent execution



The `getpid` and `getppid` functions

```
#include <sys/types.h>
#include <unistd.h>

pid_t getpid();
pid_t getppid();
```

- **`getpid()` returns the PID of the calling process**
  - The PID is guaranteed to be unique
- **`getppid()` returns the PID of the parent of the calling process**

The `sleep` function

```
#include <unistd.h>

int sleep(int seconds);
```

- **Suspends execution of the process during (at least) n seconds**
  - If a process receives a signal while it is sleeping, it may wake up before its intended time (more on this in the next classes)

- **Important note!**
  - `sleep()` can be used to try to obtain an order between processes, but it is not guaranteed
  - So, use it only for testing and debugging, **not to guarantee an execution order and/or synchronization**!

The `exit` function

```
#include <stdlib.h>

void exit(int status);
```

- **Terminates a process, returning an exit status to the OS**
  - The exit status is stored in a single byte, meaning it can hold values between 0 and 255
- **Two constants can be used, `EXIT_SUCCESS` and `EXIT_FAILURE`, to indicate success or failure in the execution**
  - 0 typically indicates successful execution
  - Nonzero values (1–255) usually signal errors or specific termination codes

Waiting for children termination

- **In general, a parent process cannot predict when any of its children will terminate**

- **Children run independently, and their terminations are asynchronous with respect to the parent's execution**
  - They don't terminate at the exact same point in time during the parent's execution each time the process runs

- **Therefore, with a synchronous method of waiting, a parent has just two choices for how it can wait**
  - It can block itself until some child terminates or otherwise changes its state
  - It can periodically check, without blocking, whether a child terminated or changed state (a topic for the next class)

The `wait` function

```
#include <sys/wait.h>

pid_t wait(int *status);
```

- **Blocks execution until any of the process's child processes terminate**
  - Returns the PID of the terminated child, or -1 in case of error
- **The `status` parameter must be a pointer to a variable that will store the child's exit status**
  - The exit status includes the child's return value and additional information provided by the OS (e.g., termination signal, abnormal exit conditions)

Example – synchronizing with `wait()`

- **What guarantees can we have regarding the output?**

```
pid = fork();

if(pid > 0){
        printf("Luke, I'm your father\n");

        /* Parent waits for child's termination */
        wait(NULL);

        printf("The child has terminated");
}
else{
        printf("I'm the child\n");
        some_code();
        exit(0);
}
```

Process exit information

- **The OS also stores some information related to the way a process terminates:**
  - The exit value
  - The signal that caused the abnormal termination (if any)
  - If the process generated a core dump
  - etc.

- **There are several macros that can be used to obtain that information from the status obtained in `wait()`**
  - **Or with `waitpid()`, which will be discussed in the next class**

Process exit information

- **`WIFEXITED(status)`**
  - Checks if the child terminated normally
- **`WEXITSTATUS(status)`**
  - Exit value (1 byte) given in the exit function
- **`WIFSIGNALED(status)`**
  - Checks if the child terminated due to a non-treated signal

Process exit information

- **`WTERMSIG(status)`**
  - Gets the number of the signal that terminated the child
- **`WCOREDUMP(status)`**
  - Checks if a core dump was generated
- **`WIFSTOPPED(status)`**
  - Checks if the child is in the STOPPED state
- **`WIFCONTINUED(status)`**
  - Checks if the process was continued

Example – process exit information

```
pid_t pid1, pid2;
int status;

pid1 = fork();

if(pid1 > 0){
    /* Parent waits for child to terminate */
    pid2 = wait(&status);

    if(WIFEXITED(status))
       printf("Parent: child %d returned %d\n", pid2, WEXITSTATUS(status));
}
else{
    sleep(5);
    exit(10);
}
```

Exercise 1

- **Implement a program that creates 2 new processes, with the following behavior:**
  - Child 1 sleeps 5 seconds, prints its PID and terminates with value 1
  - Child 2 prints its PID and the parent PID and terminates with value 2
- **Your program should enable both child processes to execute concurrently**
- **Parent should wait for children termination, printing their PIDs and exit values**